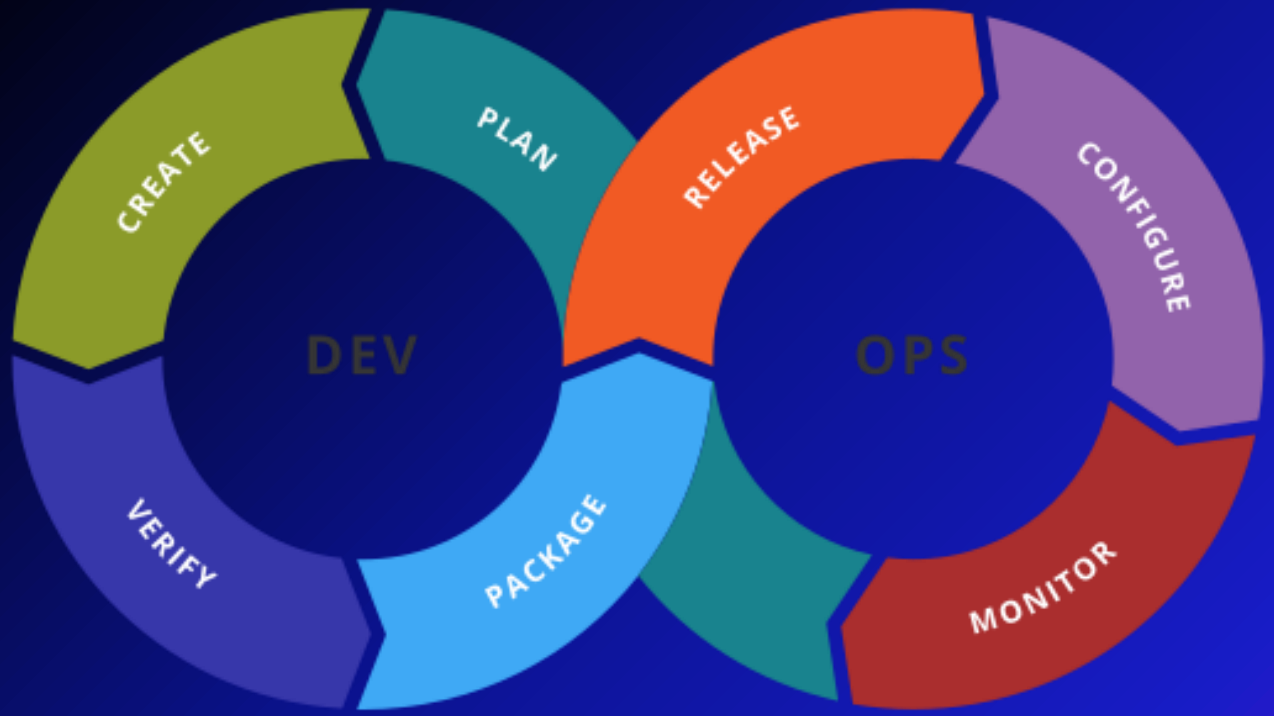
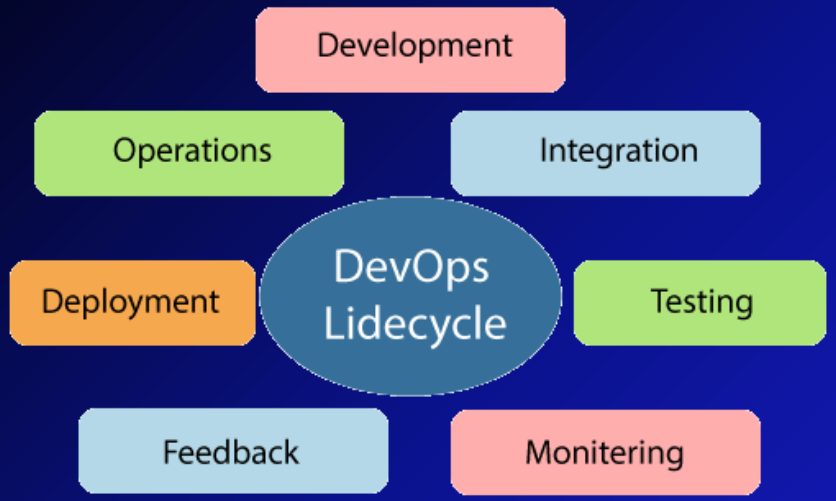
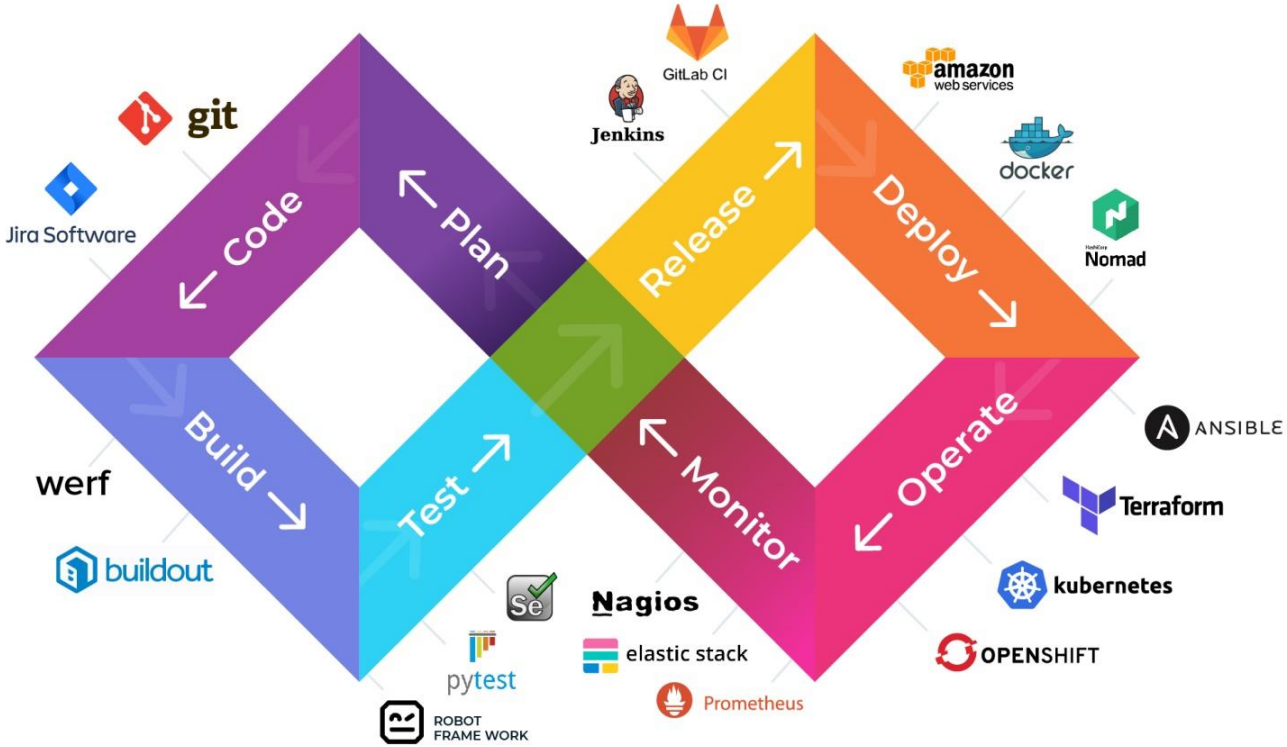


Phases & Tools



DevOps is the next evolution of agile methodologies. A cultural shift that brings development and operations teams together.



A DevOps process needs to use the right tools to address the key phases of the DevOps lifecycle.

slack

Confluence

Jira Software

Plan

Taking a page out of the agile handbook, tools that allow development and operations teams to break work down into smaller and manageable chunks for quicker deployments are required. This allows you to learn from users sooner and helps with optimizing a product based on the feedback. Look for tools that provide sprint planning, issue tracking, and allow collaboration, such as Jira.

Plan

Production-identical environments for development:

Infrastructure as code:

Source control and collaborative coding:

Build

Production-identical environments for development

While Puppet and Chef primarily benefit operations, developers use open source tools like Kubernetes and Docker to provision individual development environments. Coding against virtual, disposable replicas of production helps you get more work done.

Build

Infrastructure as code

Infrastructure as code means re-provisioning is faster than repairing – and more consistent and reproducible. It also means you can easily spin up variations of your development environment with similar configuration as production. Provisioning code can be applied and reapplied to put a server into a known baseline. It can be stored in version control. It can be tested, incorporated into CI (continuous integration), and peer-reviewed.

Build

Source control and collaborative coding

Source control tools should integrate with other tools, which allows you to connect the different parts of code development and delivery. This allows you to know if the feature’s code is running in production. If an incident occurs, the code can be retrieved to shed light on the incident.

Build

Continuous integration: Bitbucket slack

Test: mabl XRAY ZEPHYR Real-Time Test Management SAUCELABS Testing at the speed of awesome.

Deployment dashboards: Jira Software

Automated deployment: Bitbucket

Continuous integration and delivery

Continuous integration

Continuous integration is the practice of checking in code to a shared repository several times a day, and testing it each time. That way, you automatically detect problems early, fix them when they're easiest to fix, and roll out new features to your users as early as possible.

Continuous integration and delivery

Test

Testing tools span many needs and capabilities, including exploratory testing, test management, and orchestration. However, for the DevOps toolchain, automation is an essential function. Automated testing pays off over time by speeding up your development and testing cycles in the long run. And in a DevOps environment, it’s important for another reason: awareness.

Deployment dashboards

One of the most stressful parts of shipping software is getting all the change, test, and deployment information for an upcoming release into one place. The last thing anyone needs before a release is a long meeting to report on status.

This is where release dashboards come in.

Automated deployment

There’s no magic recipe for automated deployment that will work for every application and IT environment. But converting operations’ runbook into a cmd-executable script using Ruby or bash is a common way to start. Good engineering practices are vital. Use variables to factor out host names – maintaining unique scripts or code for each environment is no fun (and misses half the point anyway). Create utility methods or scripts to avoid duplicated code. And peer review your scripts to sanity-check them.

Continuous integration and delivery

Application and server performance monitoring:

Incident, change and problem tracking:

Operate

Application and server performance monitoring

Manually “topping” a box or hitting your API with a test is fine for spot-checking. But to understand trends and the overall health of your application (and environments), you need software that is listening and recording data 24/7. Ongoing observability is a key capability for successful DevOps teams.

Operate

Incident, change and problem tracking

he keys to unlocking collaboration between DevOps teams is making sure they're viewing the same work. What happens when incidents are reported? Are they linked and traceable to software problems? When changes are made, are they linked to releases? Nothing blocks Dev's collaboration with Ops more than having incidents and software development projects tracked in different systems.

Operate

Continuous Feedback

Customers are already telling you whether you've built the right thing – you just have to listen. Continuous feedback includes both the culture and processes to collect feedback regularly, and tools to drive insights from the feedback. Continuous feedback practices include collecting and reviewing NPS data, churn surveys, bug reports, support tickets, and even tweets. In a DevOps culture, everyone on the product team has access to user comments because they help guide everything from release planning to exploratory testing sessions.

Continuous Feedback

Youness Beriane Badi

ys_beriane_badi@outlook.com
linkedin.com/in/younessberianebadi

“DevOps shouldn’t be bought from a single vendor, but built”

Source: www.atlassian.com/devops/devops-tools